

Nonparametric Correlation - Examples in R

B.A. Hartlaub and A.S. Wagaman

Salary Data for Example

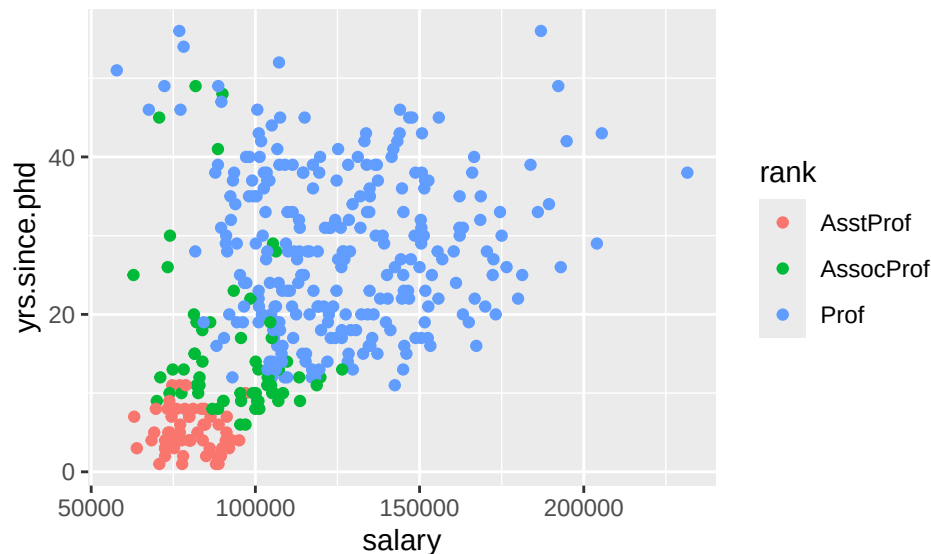
```
data(Salaries)
# ?Salaries # remove the starting comment to learn about the data
names(Salaries)
```

```
## [1] "rank"          "discipline"     "yrs.since.phd"  "yrs.service"
## [5] "sex"           "salary"
```

You can read the help file to refresh your memory about the data. There are three categorical and three quantitative variables. We will use the quantitative variables to examine correlation computations.

Here is one scatterplot of interest, color-coded by rank of the professor - assistant, associate, or full professor.

```
ggplot(data = Salaries, aes(x = salary, y = yrs.since.phd, color = rank)) +
  geom_point()
```



Do you see any unusual patterns in the plot? What kind of association is visible in the plot?

Parametric Computations - Pearson's (product moment correlation coefficient)

To begin, we want to compute the correlation using a parametric method. This is very easy.

```
# computes correlation between the two supplied variables
cor(yrs.since.phd ~ salary, data = Salaries)
```

```
## [1] 0.4192311
# computes correlation AND performs test
# note that cor.test also accepts the formula format and usual data argument
cor.test(yrs.since.phd ~ salary, data = Salaries)

##
## Pearson's product-moment correlation
##
## data: yrs.since.phd and salary
## t = 9.1775, df = 395, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.3346160 0.4971402
## sample estimates:
## cor
## 0.4192311

with(Salaries, cor.test(yrs.since.phd, salary)) # equivalent, using the *with* command

##
## Pearson's product-moment correlation
##
## data: x and y
## t = 9.1775, df = 395, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.3346160 0.4971402
## sample estimates:
## cor
## 0.4192311
```

What is the reported correlation estimate between yrs.since.phd and salary?

If there were missing observations, you might have to specify another option in the cor() function call. This might look like this:

```
# Option tells it to use all pairs it possibly can, but ignore missing values
cor(yrs.since.phd ~ salary, data = Salaries, use = "pairwise.complete.obs")

## [1] 0.4192311
```

Is the correlation significantly different from 0?

What is the 95% confidence interval for the Pearson correlation coefficient?

Here is how you can ask for a difference confidence level:

```
cor.test(yrs.since.phd ~ salary, data = Salaries, conf.level = 0.90)

##
## Pearson's product-moment correlation
##
## data: yrs.since.phd and salary
## t = 9.1775, df = 395, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 90 percent confidence interval:
## 0.3486378 0.4850946
## sample estimates:
```

```
##          cor
## 0.4192311
```

Now we will consider some nonparametric estimates.

Nonparametric Computations - Kendall's tau (Hollander, Wolfe, and Chicken 8.1-8.4)

```
# computes correlation AND performs test; note method
cor.test(yrs.since.phd ~ salary, data = Salaries, method = "kendall")
```

```
##
## Kendall's rank correlation tau
##
## data: yrs.since.phd and salary
## z = 10.038, p-value < 2.2e-16
## alternative hypothesis: true tau is not equal to 0
## sample estimates:
##          tau
## 0.3410184
```

What is the estimated value of tau (τ , if you want the Greek notation)? (Remember this is $\bar{K}$ in Hollander, Wolfe, and Chicken notation).

What test statistic is output here? Why?

Is the correlation as estimated with Kendall's tau significantly different from 0?

Is the test result for this nonparametric test consistent with that of the parametric procedure?

Note that this output did not provide a CI for us. How would we get a CI? It requires a different command in the *NSM3* package. You can specify the alpha that matches the desired confidence level, and can specify if you want the usual symmetric interval or a bound with the type argument. The default alpha is 0.05 (so a 95 percent confidence interval) and the default type is *t*, for the symmetric interval. So, if that is what you want, you can leave off those arguments.

```
# be sure the NSM3 package is loaded for this function to be available
with(Salaries, kendall.ci(yrs.since.phd, salary, alpha = 0.05, type = "t"))
```

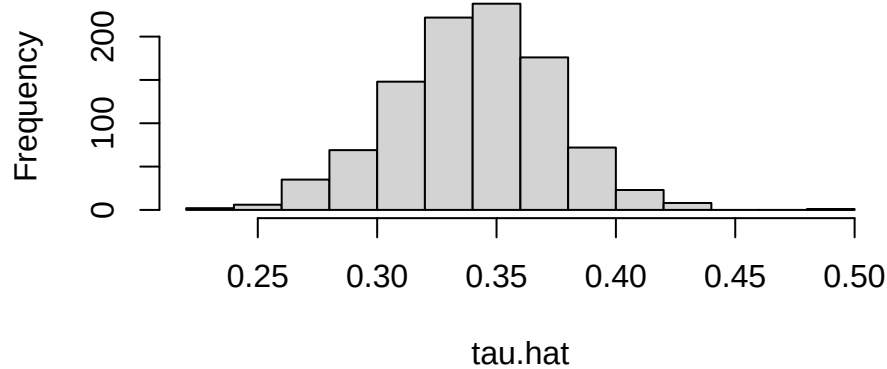
```
##
## 1 - alpha = 0.95 two-sided CI for tau:
## 0.276, 0.406
```

An alternative approach to find the CI using the bootstrap (see Hollander, Wolfe, and Chicken 8.4). The same command can be used, but additional arguments are needed. The alpha and type arguments remain and should be adjusted as needed (or you can leave them out if the default is what you want). The additional arguments are *bootstrap = T*, to tell it to use the bootstrap, and then *B*, the number of bootstrap samples. Note that recommended values of *B* are 1000 or 2000 (100 or 200 are usually considered too small for making a confidence interval).

Finally, as the bootstrap is a random process, we set a seed so that every time we run this chunk, we obtain the same CI. Without a seed (or by changing the seed), different intervals would be obtained, though they would be expected to be somewhat similar to each other. Note that running the bootstrap code can take some time.

```
set.seed(225) # can input any value you want
with(Salaries, kendall.ci(yrs.since.phd, salary, alpha = 0.05, type = "t",
                          bootstrap = T, B = 1000))
```

Histogram of tau.hat



```
##  
## 1 - alpha = 0.95 two-sided CI for tau:  
## 0.273, 0.404  
  
# if default is what you want, can remove later options like this  
set.seed(225) # we set a seed again so this interval is the same as the one above  
# without the seed again, even though the code is equivalent,  
# the interval returned could be different  
with(Salaries, kendall.ci(yrs.since.phd, salary, bootstrap = T, B = 1000))  
  
##  
## 1 - alpha = 0.95 two-sided CI for tau:  
## 0.273, 0.404
```

There are a number of other functions in R that will perform a bootstrap and you can also code it yourself, to use the same approach in other settings. Useful functions may be `do` and `resample` in the `mosaic` package. The bootstrap is a valuable statistical tool, applicable in many more situations beyond creating confidence intervals for Kendall's tau.

Critical Values for Kendall's tau

Briefly, let's look at an example where we want to obtain critical values for Kendall's tau. This section is a bit challenging. We will need these critical values in Chapter 9 (HWC) though, so we want you all to have some practice getting them. We'll use just the first five observations of the data set. (Why 5? Using 8 would introduce a tie, and we want to demonstrate this without ties, and we also want fewer computations, so we chose $5 < 8$.)

```
Salariessmall <- slice(Salaries, 1:5)  
cor.test(yrs.since.phd ~ salary, data = Salariessmall, method = "kendall")  
  
##  
## Kendall's rank correlation tau  
##  
## data: yrs.since.phd and salary  
## T = 6, p-value = 0.8167  
## alternative hypothesis: true tau is not equal to 0  
## sample estimates:
```

```
## tau
## 0.2
```

We see the estimated tau is 0.2, with $T = 6$ (this is K' , spoken as K prime), and p-value of 0.8167. How can we find associated critical values in this setting?

First, we need to make sure we understand the computations. Let's step through the computation.

```
X <- Salariessmall$yrs.since.phd
Y <- Salariessmall$salary
ind <- sort(X,index.return=TRUE) # store sorted index
Xsort <- X[ind$ix] # use sorted index to sort X's
Ysort <- Y[ind$ix] # use sorted index to keep Y's with their corresponding X
Xsort
```

```
## [1] 4 19 20 40 45
```

```
Ysort
```

```
## [1] 79750 139750 173200 141500 115000
```

Now, we want to count the number of concordant pairs of Y's. Remember, the idea in concordance is that we've sorted the X's from smallest to largest, and want to see if the Y's are arranged preserving this order as well (or at least, how much is it preserved?). If we look at the sorted Y's, a pair is concordant if the first value is smaller than the second value, looking left to right, and we have to do all pairs of values.

Try this looking at Ysort (remember to go left to right). For 79750 paired with all the others, we count 4 concordant pairs (all pairs are concordant). For 139750 paired with the others, we count 2 concordant pairs. It is more than the final Y of 115000. For the remaining two, there are no more concordant pairs. Thus, we have 6 total concordant pairs, which is why we had a K' of 6. This matches the $T=6$ reported. Having K' is just part of the puzzle though. How can we get K , the Kendall statistic, from K' ? Looking at concordant pairs is often easier than getting K directly, which is why we did it, but we want K sometimes, so we need to be able to get it.

Recall that, when considering the number of concordant pairs K' and the number of discordant pairs K'' , we know that

$$K' + K'' = n(n-1)/2.$$

It can be shown that (Comment 4 in 8.1 in Hollander, Wolfe, and Chicken) K can be expressed as a function of K' , as

$$K = 2K' - n(n-1)/2.$$

So, if K' is 6, then

```
n <- 5
Kprime <- 6
K <- 2*Kprime-(n*(n-1)/2); K # K=2
```

```
## [1] 2
```

If you have K and want to solve for K' , we just rearrange this equation.

$$K' = (K + n * (n-1)/2)/2$$

```
n <- 5
K <- 2
Kprime <- (K+n*(n-1)/2)/2; Kprime # yields K'=2
```

```
## [1] 6
```

The p-value from R from the test we performed was 0.8167. Let's make sure we can replicate that. Note that this function requires the estimated tau as the input, not K or K' .

```
# estimated tau was 0.2 for our small sample  
pKendall(0.2, N = 5) # says P(Kbar <= 0.2) = 0.758333
```

```
## [1] 0.7583333
```

```
# however, this isn't what we want  
# we want the probability associated with 0.2 to be included in the tail  
# so we'd need the next value possible, below 0.2, and that's hard to know  
# To get our p-value then, what do we do?  
# we need to rely on symmetry (to avoid issues with discrete distributions)  
2*pKendall(-0.2, N = 5) # use symmetry, lower-tailed value to start, then double
```

```
## [1] 0.8166667
```

How would we find the cutoff that would give an alpha for a two-tailed test of 0.10? Well, that would put 0.05 in each tail, so we can use:

```
qKendall(0.05, N = 5)
```

```
## [1] -0.6
```

This suggests values ≥ 0.6 , or ≤ -0.6 should be considered as the cutoffs. However, we can check to see how the alpha compares:

```
2*pKendall(-0.6, N = 5)
```

```
## [1] 0.2333333
```

The alpha would really be 0.23333, which is much larger than 0.10. The challenge here is that it's not easy to figure out the NEXT possible value of the tau estimate, but it can be done. Let's take a look at how we'd do that.

```
Kbar <- -0.6 # current Kbar  
n <- 5  
K <- Kbar*n*(n-1)/2 # solve for K  
Kprime <- (K+n*(n-1)/2)/2; Kprime # solve for Kprime
```

```
## [1] 2
```

The next possible value would have a Kprime value adjusted by 1 down (for when you are in the lower tail) or 1 up (for when you are in the upper tail). So, we consider what happens with $K' = 1$ in this example, shifting down from 2, as we are in the lower tail.

```
n <- 5  
Kprime <- 1  
K <- 2*Kprime-(n*(n-1)/2); K
```

```
## [1] -8
```

```
Kbar <- 2*K/(n*(n-1)); Kbar
```

```
## [1] -0.8
```

The next possible value is -0.8, which gives an alpha of 0.0833 for the two-sided test.

```
2*pKendall(-0.8, N = 5)
```

```
## [1] 0.08333333
```

How could we have found this value faster? We could look up a series of quantiles around 0.05 and 0.95, to see the possible values of the estimated tau.

```
qKendall(c(0.01, 0.02, 0.04, 0.05, 0.08, 0.10, 0.12, 0.14), N = 5)
```

```
## [1] -0.8 -0.8 -0.8 -0.6 -0.6 -0.6 -0.4 -0.4
```

Here's how this would work in an upper-tailed setting:

```
qKendall(c(0.86, 0.88, 0.90, 0.92, 0.95, 0.96, 0.98, 0.99), N = 5)
```

```
## [1] 0.4 0.4 0.6 0.6 0.6 0.8 0.8 0.8
```

Note that as N gets larger, you need more and more possible quantiles here in order to see the different possible values. However, this can be easier (even with that adjustment) than doing the computations though.

Nonparametric Computations - 8.5 - Spearman's rho

```
# computes rank statistic AND performs test; note method
cor.test(yrs.since.phd ~ salary, data = Salaries, method = "spearman")
```

```
## Warning in cor.test.default(x = mf[[1L]], y = mf[[2L]], ...): Cannot compute
## exact p-value with ties
```

```
##
```

```
## Spearman's rank correlation rho
```

```
##
```

```
## data: yrs.since.phd and salary
```

```
## S = 5468989, p-value < 2.2e-16
```

```
## alternative hypothesis: true rho is not equal to 0
```

```
## sample estimates:
```

```
## rho
```

```
## 0.4755676
```

What is the value of the rank statistic (r_s , listed as the estimate of rho)? Remember that this cannot exactly be used as a correlation to compare to r and $\bar{K}$. However, you can still consider positive values indicative of positive correlation, and similar for negative values. (Details and alternatives in HWC.)

What is the notation for the test statistic output by this function for the Spearman method?

Is the correlation significantly different from 0 based on this procedure?

Is the test result for this nonparametric test consistent with that of the parametric procedure?

Note that because the r_s cannot be used as a correlation estimate to compare to r and $\bar{K}$, CIs for this value are not made.

Critical Values for Spearman's rho

Briefly, we consider some examples finding cutoffs and p-values for Spearman's rho.

Suppose $n = 8$. Then the possible values of Spearman's rho can be found with:

```
# r is used for n, an unusual notation choice in this function
round(qSpearman(seq(from = 0.01, to = 0.99, by = 0.01), r = 8), 2)
```

```
## [1] -0.76 -0.71 -0.67 -0.62 -0.60 -0.57 -0.55 -0.52 -0.50 -0.48 -0.45 -0.43
```

```
## [13] -0.43 -0.40 -0.38 -0.38 -0.36 -0.33 -0.33 -0.31 -0.31 -0.29 -0.29 -0.26
```

```
## [25] -0.24 -0.24 -0.21 -0.21 -0.19 -0.19 -0.17 -0.17 -0.14 -0.14 -0.12 -0.12
```

```
## [37] -0.10 -0.10 -0.10 -0.07 -0.07 -0.05 -0.05 -0.02 -0.02 0.00 0.00 0.02
```

```
## [49] 0.02 0.02 0.05 0.05 0.07 0.07 0.10 0.10 0.12 0.12 0.14 0.14
## [61] 0.17 0.17 0.17 0.19 0.19 0.21 0.21 0.24 0.24 0.26 0.26 0.29
## [73] 0.29 0.31 0.31 0.33 0.36 0.36 0.38 0.38 0.40 0.40 0.43 0.45
## [85] 0.45 0.48 0.50 0.50 0.52 0.55 0.57 0.60 0.62 0.64 0.67 0.69
## [97] 0.74 0.79 0.83
```

We note that this distribution should be symmetric around 0, and it is, for the most part, until the extreme tails. We will still rely on symmetry (see below - it actually is symmetric, but the code authors appear to have already dealt with the discrete issue we usually face. Either that, or there is something amiss.)

Here, the distribution still has some discrete aspects (you see values can repeat as quantiles), but it's more continuous than other situations we have seen. It's also fairly clear that it would be hard to determine what the +1/-1 (next value) would be in order to adjust. Thus, for this test procedure, we will not worry about adjusting for the discreteness in the distribution. This will mean that results are APPROXIMATE, not exact, however.

So, if we want to find the upper-tailed cutoff for an $\alpha=0.05$ level test when $n=12$, we would use symmetry, and find the cutoff for a lower-tailed test with:

```
qSpearman(0.05, r = 12)
```

```
## [1] -0.493007
```

This yields -0.49301. Therefore, our r_s cutoff for the upper-tail would be 0.49301. To determine the associated α (which may go over 0.05), we check:

```
pSpearman(-0.49301, r = 12)
```

```
## [1] 0.04825218
```

This is 0.048252, so it does not go over 0.05. This is great (and may be part of how the function is coded).

Checking the upper-tail probability is hard because we don't know what the next value down (that shift down 1) is. And if we compute $1 - P(r_s \leq 0.49301)$, that's not the correct probability. So, we recommend using symmetry and lower-tailed probabilities to assess your α levels and cutoffs.

What if we wanted the p-value for an upper-tailed test where $n=8$ and we observed $r_s = 0.59524$? Using symmetry, our p-value is found with

```
pSpearman(-0.59524, r = 8)
```

```
## [1] 0.06614583
```

This value (0.59524) was rounded to 0.60 in the lists of possible r_s values previously examined.

Note for test output

Most of our examples were two-sided here (for all the test output). Bear in mind you can submit a directed alternative hypothesis. Please be sure you are comfortable obtaining one-sided results from two-sided results, and vice versa.

Data Sources

Salaries data from carData package

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.